Chapter 30 | Exercise solutions with Python

Decisions and Embodied Action

This notebook solves Exercises 5–8. It computes returns-to-go, checks how a policy handles a requested return absent from the log, compares rollouts in two layouts, and changes the mix of demonstrations.

The examples run locally in a small deterministic grid. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch30_decisions"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check the return routine

Start at the end of the trajectory with rewards $(0, 2, -1, 4)$. The last action earns 4, so its return-to-go is 4. One step earlier, the action costs 1 and the later action earns 4, leaving 3. Moving back two more steps adds 2 and then 0. In chronological order, the four returns are $(5, 5, 3, 4)$.

In general, the return-to-go G_t includes the reward r_t from the current action and all rewards after it. With $G_T = 0$ after the final action, the backward calculation uses $G_t = r_t + G_{t+1}$. When we read the trajectory forward, the same rule gives $G_{t+1} = G_t - r_t$. After the penalty at step 2, subtracting -1 gives $G_3 = 3 - (-1) = 4$.

At step 0 the return includes the whole trajectory, so it equals the sum $0 + 2 - 1 + 4 = 5$. A trajectory with just one reward of -3 has only one return, which also equals its total of -3. The code checks both full vectors, compares their first entries with Python's `sum`, and verifies the forward rule through the terminal zero.

```
from examples.ch30_decisions.model import returns_to_go

reward_cases = [
    ([0, 2, -1, 4], [5, 5, 3, 4]),
    ([-3], [-3]),
]
for rewards, expected_returns in reward_cases:
    returns = returns_to_go(rewards)
    total = sum(rewards)
    next_returns = np.append(returns[1:], 0)

    np.testing.assert_array_equal(returns, expected_returns)
    assert returns[0] == total
    np.testing.assert_array_equal(next_returns, returns - rewards)

    print(f"Rewards = {rewards}")
    print(f"  Returns-to-go = {returns.tolist()}")
    print(f"  First return = {returns[0]:g}, total reward = {total}")
```

OUTPUT

```
Rewards = [0, 2, -1, 4]
  Returns-to-go = [5.0, 5.0, 3.0, 4.0]
  First return = 5, total reward = 5
Rewards = [-3]
  Returns-to-go = [-3.0]
  First return = -3, total reward = -3
```

6. Test an unsupported target

The log starts every demonstration at $(0, 0)$. The three failed episodes each have a return of -1 there, and the successful episode has a return of $+1$. Fitting the second table stores an action for each observed pair of position and return-to-go. At the initial position a request for $+2$ has no entry. With -1, the same table has the action `R` for right.

Keep the original grid and run the conditioned policy with both targets. The trace records the initial state even when the first table lookup fails. Each action that executes adds one resulting state and one reward.

```
from examples.ch30_decisions.model import START, fit_count_policies, rollout
from examples.ch30_decisions.run_examples import TRAJECTORIES

_, return_policy = fit_count_policies(TRAJECTORIES)
assert (START, 2) not in return_policy
assert return_policy[START, -1] == "R"

unsupported = rollout(return_policy, target_return=2)
assert unsupported == {
    "states": [(0, 0)],
    "actions": [],
    "rewards": [],
    "remaining_targets": [2],
    "total_reward": 0,
    "stop_reason": "unsupported",
}

hit_trap = rollout(return_policy, target_return=-1)
assert hit_trap == {
    "states": [(0, 0), (0, 1)],
    "actions": ["R"],
    "rewards": [-1],
    "remaining_targets": [-1, 0],
    "total_reward": -1,
    "stop_reason": "trap",
}

fields = ("stop_reason", "states", "actions", "rewards", "total_reward")
for target, trace in [(2, unsupported), (-1, hit_trap)]:
    print(f"Requested return = {target:+d}")
    for field in fields:
        print(f"  {field} = {trace[field]}")
```

OUTPUT

```
Requested return = +2
  stop_reason = unsupported
  states = [(0, 0)]
  actions = []
  rewards = []
  total_reward = 0
Requested return = -1
  stop_reason = trap
  states = [(0, 0), (0, 1)]
  actions = ['R']
  rewards = [-1]
  total_reward = -1
```

With $+2$, the policy stops with `unsupported` at the first lookup. The visited state list contains just $(0, 0)$. The action and reward lists are empty, so the total reward is 0. With -1, the policy chooses `R`. The simulator moves to $(0, 1)$, records reward -1, and ends the episode with `trap`. The first stop comes from a missing entry in the learned table. The second comes from an executed action reaching a terminal cell.

7. Inspect the shifted rollout

Keep the fitted policy and the starting target +1 fixed. The original grid has its trap at $(0, 1)$ and the shifted grid moves it to $(1, 1)$, a cell on the route to the goal. Run the chapter's example module to see what the move changes.

The subprocess uses the notebook's Python interpreter and runs from the repository root so Python can find the module. The JSON file lives in a temporary directory and the parsed result stays available as a Python dictionary.

```
import json
import subprocess
from tempfile import TemporaryDirectory

with TemporaryDirectory() as temporary_directory:
    output_path = Path(temporary_directory) / "ch30.json"
    subprocess.run(
        [
            sys.executable,
            "-m",
            "examples.ch30_decisions.run_examples",
            "--output",
            str(output_path),
        ],
        cwd=repo_root,
        check=True,
        stdout=subprocess.PIPE,
    )
    result = json.loads(output_path.read_text(encoding="utf-8"))
grid = result["grid"]

original_grid_run = grid["return_conditioned"]
shifted_grid_run = grid["shifted_return_conditioned"]
```

Each trace records the starting position as its first state. With zero-based indices, `actions[i]` leaves `states[i]` and arrives at `states[i + 1]` with reward `rewards[i]`. Print both transitions using one-based step numbers, then compare the resulting state and reward at each step the runs have in common.

```
assert grid["original_trap"] == [0, 1]
assert grid["shifted_trap"] == [1, 1]

for label, run in (
    ("Original grid", original_grid_run),
    ("Shifted grid", shifted_grid_run),
):
    assert run["states"][0] == [0, 0]
    assert len(run["states"]) == len(run["actions"]) + 1
    print(label)
    for index, action in enumerate(run["actions"]):
        state = tuple(run["states"][index])
        next_state = tuple(run["states"][index + 1])
        reward = run["rewards"][index]
        print(
            f"  {index + 1}. {state} --{action}--> {next_state}"
            f"  reward {reward:+d}"
        )
    print(f"  Stop at {run['stop_reason']}")
    print()

assert original_grid_run["actions"] == ["D", "R", "R", "U"]
assert shifted_grid_run["actions"] == ["D", "R"]
assert original_grid_run["stop_reason"] == "goal"
assert shifted_grid_run["stop_reason"] == "trap"
```

OUTPUT

```
Original grid
  1. (0, 0) --D--> (1, 0)  reward +0
  2. (1, 0) --R--> (1, 1)  reward +0
  3. (1, 1) --R--> (1, 2)  reward +0
  4. (1, 2) --U--> (0, 2)  reward +1
  Stop at goal

Shifted grid
  1. (0, 0) --D--> (1, 0)  reward +0
  2. (1, 0) --R--> (1, 1)  reward -1
  Stop at trap
```

```
original_outcomes = zip(
    original_grid_run["states"][1:], original_grid_run["rewards"],
    strict=True,
)
shifted_outcomes = zip(
    shifted_grid_run["states"][1:], shifted_grid_run["rewards"],
    strict=True,
)
for step, (original, shifted) in enumerate(
    zip(original_outcomes, shifted_outcomes), start=1
):
    if original != shifted:
        break
else:
    raise AssertionError("Expected the rollout outcomes to differ.")

assert step == 2
assert original == ([1, 1], 0)
assert shifted == ([1, 1], -1)
assert original_grid_run["remaining_targets"] == [1, 1, 1, 1, 0]
assert shifted_grid_run["remaining_targets"] == [1, 1, 2]
print(f"First different outcome at step {step} (index {step - 1}).")
```

```
OUTPUT
First different outcome at step 2 (index 1).
```

The first difference is the reward from the second action. Both agents move right into $(1, 1)$. That cell gives 0 on the original grid and -1 on the shifted grid, where entering it ends the episode. Subtracting the reward leaves a target of $1 - 0 = 1$ on the original grid and $1 - (-1) = 2$ on the shifted grid. The latter is recorded in the trace at termination.

The policy looks up an action using the current position and the remaining target. In both runs it starts at $((0, 0), +1)$ and chooses `D`. That move goes to $(1, 0)$ and gives reward 0 on both grids. The next lookup is therefore $((1, 0), +1)$ in both runs, so the policy chooses `R` before receiving the reward for that move.

8. Vary the demonstrations

On the original grid, each trap demonstration starts with `R` and each goal demonstration starts with `D`. The chapter's original log has three trap demonstrations and one goal demonstration. Keep the same two routes and reverse how often they appear. Count the logged actions at the start and use the chapter's function to fit both policy tables on each log.

```
from collections import Counter

from examples.ch30_decisions.model import (
    START,
    fit_count_policies,
    rollout,
)
from examples.ch30_decisions.run_examples import (
    GOAL_TRAJECTORY,
    TRAJECTORIES,
    TRAP_TRAJECTORY,
)

rebalanced_log = (TRAP_TRAJECTORY,) + (GOAL_TRAJECTORY,) * 3
original_state, original_conditioned = fit_count_policies(
    TRAJECTORIES
)
rebalanced_state, rebalanced_conditioned = fit_count_policies(
    rebalanced_log
)

print("Log          R count  D count  State-only  Conditioned (+1)")
for label, log, state_policy, conditioned_policy, expected in (
    (
        "Original", TRAJECTORIES, original_state,
        original_conditioned, {"R": 3, "D": 1},
    ),
    (
        "Rebalanced", rebalanced_log, rebalanced_state,
        rebalanced_conditioned, {"R": 1, "D": 3},
    ),
):
    start_counts = Counter(
        action
        for trajectory in log
        for state, action, _ in trajectory
        if state == START
    )
    assert start_counts == expected
    state_choice = state_policy[START]
    conditioned_choice = conditioned_policy[(START, 1)]
    print(
        f"{label:12} {start_counts['R']:7} {start_counts['D']:8}"
        f" {state_choice:^11} {conditioned_choice:^16}"
    )

assert original_state[START] == "R"
assert rebalanced_state[START] == "D"
assert original_conditioned[(START, 1)] == "D"
assert rebalanced_conditioned[(START, 1)] == "D"
```

OUTPUT

```
Log          R count  D count  State-only  Conditioned (+1)
Original           3        1      R             D
Rebalanced         1        3      D             D
```

Now run both fitted policies on the original grid. The simulator uses the original trap by default. For the conditioned policy, pass `target_return=1` so that it follows the decisions recorded with a remaining target of +1.

```
original_state_run = rollout(original_state)
original_target_run = rollout(original_conditioned, target_return=1)
rebalanced_state_run = rollout(rebalanced_state)
rebalanced_target_run = rollout(rebalanced_conditioned, target_return=1)

assert original_state_run["actions"] == ["R"]
assert original_state_run["stop_reason"] == "trap"
assert original_state_run["total_reward"] == -1

print("Log          Policy            Actions   Stop   Total reward")
for label, policy_name, run in (
    ("Original", "State-only", original_state_run),
    ("Original", "Conditioned (+1)", original_target_run),
    ("Rebalanced", "State-only", rebalanced_state_run),
    ("Rebalanced", "Conditioned (+1)", rebalanced_target_run),
):
    actions = " ".join(run["actions"])
    print(
        f"{label:12} {policy_name:17} {actions:9} "
        f"{run['stop_reason']:6} {run['total_reward']:+d}"
    )

for run in (
    original_target_run, rebalanced_state_run, rebalanced_target_run
):
    assert run["actions"] == ["D", "R", "R", "U"]
    assert run["states"] == [(0, 0), (1, 0), (1, 1), (1, 2), (0, 2)]
    assert run["stop_reason"] == "goal"
    assert run["total_reward"] == 1
```

OUTPUT

```
Log          Policy            Actions   Stop   Total reward
Original     State-only        R         trap   -1
Original     Conditioned (+1)  D R R U   goal   +1
Rebalanced   State-only        D R R U   goal   +1
Rebalanced   Conditioned (+1)  D R R U   goal   +1
```

The state-only table pools all visits to the start. In the original log its majority action is `R` by three votes to one, so it enters the trap immediately. In the new log its majority action is `D` by the same margin. At the later cells on the goal route, the log contains only the demonstrated goal actions, so it completes `D, R, R, U` and earns +1.

The conditioned table counts the two kinds of start visits under separate keys. The `D` votes have return-to-go +1 and the `R` votes have return-to-go −1. Repeating the goal demonstration adds votes for the same action at each key on the goal route, so the +1 rollout reaches the goal with either log.